

Lecture 3 - Introduction to Supervised Learning with (Polynomial) Linear Regression, Code Implementation

Linear Regression with Polynomial Features (or *Polynomial Regression*)

Let's begin by considering the polynomial curve fitting example in the first chapter of the Bishop textbook.

- A regression task is a supervised learning task.
- The target labels are continuous.
- Linear regression is *linear* on the parameters, that is, linear regression is a linear combination of features (which may not be linear themselves).
- If the features are extracted with polynomial basis function, we call the linear regression model a **polynomial regression** model.

Polynomial Regression is a type of linear regression that uses a special set of *features* - polynomial features.

Step 1 - Input Space

Suppose we are given a training set comprising of N observations of $\mathbf{x}$, $\mathbf{x} = [x_1, x_2, \dots, x_N]^T$, and its corresponding target outputs $\mathbf{t} = [t_1, t_2, \dots, t_N]^T$, where sample x_i has the target output label t_i .

So, we want to learn the *true* function mapping f such that $\mathbf{t} = f(\mathbf{x}, \mathbf{w})$, where $\mathbf{w}$ are parameters of the model.

- Note that both the training data and target outputs can be noisy. Sometimes the target outputs can be mislabeled, so it is important to apply
- We generally organize data into *matrices* and *vectors*. Not only is it a common way to organize the data, but it allows us to easily apply algebraic operations during analysis. It also makes it much simpler when it comes to code implementation!

- In most engineering textbooks and in this course, **vectors** are defined as *column vectors*. This is why we write $\mathbf{x} = [x_1, x_2, \dots, x_N]^T$.

Step 2 - Feature Extraction

Let's consider *polynomial features* for each data point x_i , $\phi(x_i)$. We define $\phi(\cdot)$ as the **polynomial basis function** of order M : $\phi(x_i) = [x_i^0, x_i^1, x_i^2, \dots, x_i^M]^T$.

- Other features can be extracted. These can be other basis functions, but can also be extracted by constructing custom coding scripts. Other **basis functions** include:
 - Polynomials Basis functions: $\phi_j(x) = x^j$
 - Radial Basis functions: $\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2s^2}\right)$
 - Sigmoidal basis function: $\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right)$, where $\sigma(a)$ is the logistic sigmoid function defined by $\sigma(a) = \frac{1}{1+\exp(-a)}$
 - Fourier Basis functions, which leads to an expansion in sinusoidal functions. Each basis function represents a specific frequency and has infinite spatial extent
 - Wavelets Basis Functions, representing both space and frequency

```
In [7]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
```

```
In [6]: import warnings
warnings.filterwarnings("ignore")

# Basis Functions
x=np.linspace(-1,1,1000)

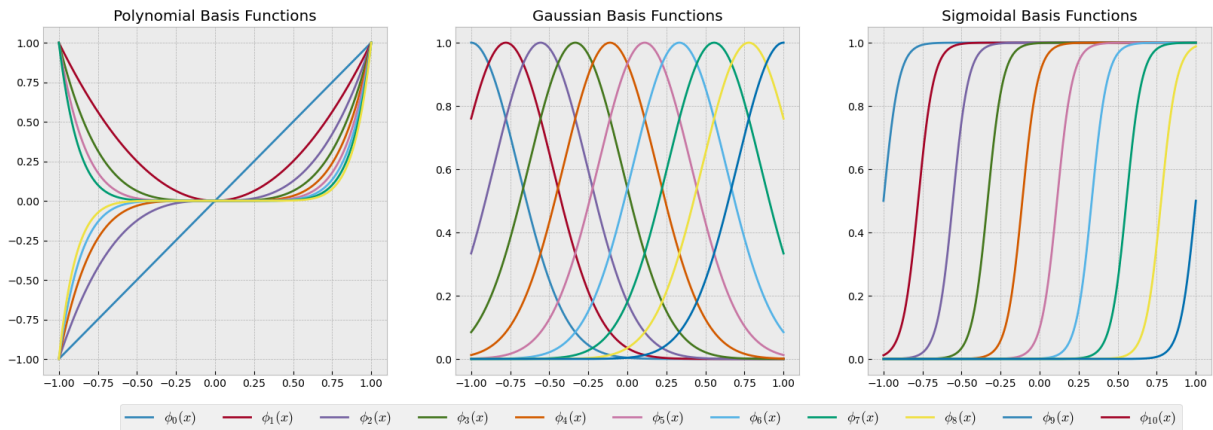
fig = plt.figure(figsize=(20,6))
plt.subplot(1,3,1)
polynomials=np.array([x**i for i in range(1,10)]).T # Polynomial Basis Functions
plt.plot(x, polynomials); plt.title('Polynomial Basis Functions')

plt.subplot(1,3,2)
sig=0.3 #fixed standard deviation for each Gaussian basis function
polynomials=np.array([np.exp(-(x-i)**2/(2*sig**2)) for i in np.linspace(-1,1,10)]).T
plt.plot(x, polynomials); plt.title('Gaussian Basis Functions')

plt.subplot(1,3,3)
```

```
sig=0.05 #fixed standard deviation for each sigmoidal basis function
polynomials=np.array([1/(1+np.exp(-(x-i)/sig)) for i in np.linspace(-1,1,10)])
plt.plot(x, polynomials); plt.title('Sigmoidal Basis Functions')

legend_labels=['$\phi_0(x)$', '$\phi_1(x)$', '$\phi_2(x)$', '$\phi_3(x)$', '$\phi_4(x)$', '$\phi_5(x)$', '$\phi_6(x)$', '$\phi_7(x)$', '$\phi_8(x)$', '$\phi_9(x)$', '$\phi_{10}(x)$']
fig.legend(legend_labels, loc = (0.1, 0), ncol=11, fontsize=13);
```



Feature Space

The set of features drawn by the transformation

$$\begin{aligned} \phi: \mathbb{R} &\rightarrow \mathbb{R}^{M+1} \quad x \mapsto [\phi_0(x), \phi_1(x), \phi_2(x), \dots, \phi_M(x)] \end{aligned}$$

is often called the **feature space**. When we write a linear regression with respect to a set of basis functions, the regression model is linear in the *feature space*.

In this case, the dimensionality of the feature space is $M+1$. This is often referred to as the *model order*.

- The **feature matrix** $\mathbf{X}$ is a matrix containing all feature mappings for all samples $\{x_i\}_{i=1}^N$ and can be written as:

$$\mathbf{X} = \begin{bmatrix} \phi(x_1)^T & \phi(x_2)^T & \dots & \phi(x_N)^T \\ 1 & x_1 & x_1^2 & \dots & x_1^M \\ 1 & x_2 & x_2^2 & \dots & x_2^M \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^M \end{bmatrix} \in \mathbb{R}^{N \times (M+1)}$$

where each row is a feature representation of a data point x_i .

- Our goal is to find the mapping from the feature input data $\mathbf{X}$ to the desired output values $\mathbf{t}$.

Suppose the data actually comes from some **unknown hidden function**, that takes in the data points $\mathbf{x}$ with some parameters $\mathbf{w}$ and produces the desired values $\mathbf{t}$, i.e. $\mathbf{t} = f(\mathbf{x}, \mathbf{w})$.

- We do not know anything about the function f . If we knew the hidden function, we would not need to learn the *mapping* - we would already know it. However, since we do not know the true underlying function, we need to do our best to estimate from the examples of input-output pairs that we have.

Step 3 - Model Selection or Mapping

Let's assume that the desired output values are a *linear combination* of the feature input space, i.e., the **polynomial function**

$$\begin{aligned} t \sim y(x, \mathbf{w}) &= w_0x^0 + w_1x^1 + w_2x^2 + \dots + w_Mx^M = \\ \sum_{j=0}^M w_jx^j &= \mathbf{X} \mathbf{w} \end{aligned}$$

- This means that for every paired training data point $\{(x_i, t_i)\}_{i=1}^N$, we can model the output value as

$$\begin{aligned} t_i \sim y(x_i, \mathbf{w}) &= w_0x_i^0 + w_1x_i^1 + \\ w_2x_i^2 + \dots + w_Mx_i^M \end{aligned}$$

- Although the polynomial function $y(x, \mathbf{w})$ is a nonlinear function of x , it is a **linear function of the parameters $\mathbf{w}$** . Functions, such as the polynomial, which are linear in the unknown parameters have important properties and are called *linear models*.

The **linear basis model** for regression takes linear combinations of deterministic nonlinear functions of the input variables

$$\begin{aligned} t \sim y(x, \mathbf{w}) &= w_0 + \sum_{j=1}^M w_j \phi_j(x) \end{aligned}$$

- The parameter w_0 allows for any fixed offset in the data and is sometimes called a *bias* parameter.
- It is often convenient to include an additional dummy *basis function* $\phi_0(x) = 1$ so that

$$\begin{aligned} t \sim y(x, \mathbf{w}) &= \sum_{j=0}^M w_j \phi_j(x) = \\ \mathbf{X} \mathbf{w} \end{aligned}$$

where $\mathbf{w} = [w_0, w_1, \dots, w_M]^T$, $\mathbf{\phi} =$

$[\phi_0, \phi_1, \dots, \phi_M]^T$ and $\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^M \\ 1 & x_2 & x_2^2 & \dots & x_2^M \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^M \end{bmatrix}$.

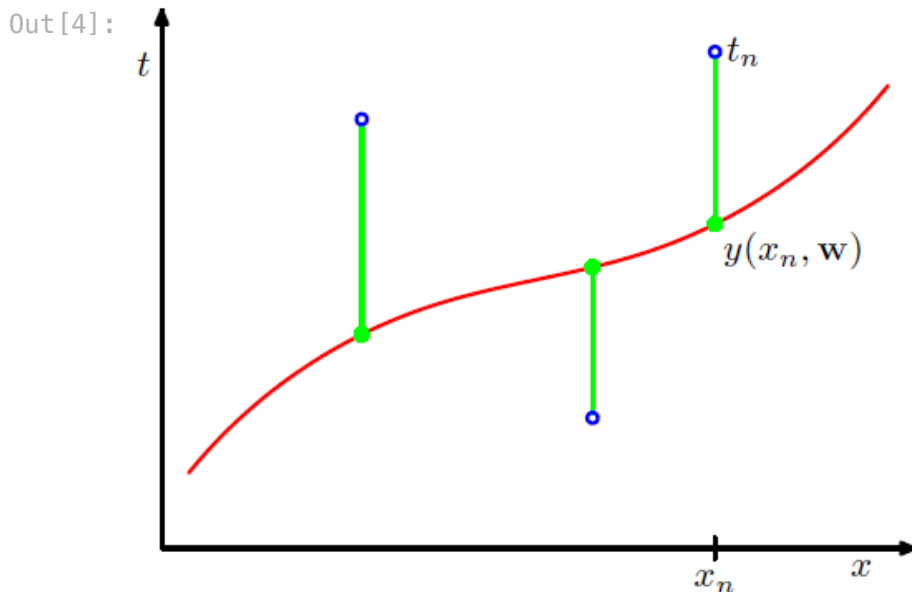
The values of the coefficients $\mathbf{w}$ will be determined by *fitting* the polynomial regression model to the training data (or simply **training the model**).

This can be done by minimizing an **objective function** (also defined as **cost function**, **error function**, or **loss function**) that measures the *misfit* between the function $y(x, \mathbf{w})$, for any given value of $\mathbf{w}$, and the training set data points $\{(x_i, t_i)\}_{i=1}^N$.

- What is the model's *objective* or goal?

```
In [4]: from IPython.display import Image
Image('figures/LeastSquares.png', width=400)

#Source: Christopher M. Bishop. Pattern recognition and machine learning. Sp
```



One simple choice for fitting the model is to consider the error function given by the sum of the squares of the errors between the predictions $y(x_i, \mathbf{w})$ for each data point x_i and the corresponding target values t_i , so that we minimize

$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{2} \sum_{n=1}^N \left(t_n - y(x_n, \mathbf{w}) \right)^2 \\ &= \frac{1}{2} \sum_{n=1}^N \left(t_n - \sum_{j=0}^M w_j x_n^j \right)^2 \\ &= \frac{1}{2} \sum_{n=1}^N \left(t_n - \phi(\mathbf{x}_n)^T \mathbf{w} \right)^2 \\ &= \frac{1}{2} \left\| \mathbf{t} - \mathbf{X} \mathbf{w} \right\|_2^2 \end{aligned}$$

Other objective functions can be considered, for example, the absolute value of the absolute error is often used for sparsity concerns. But we will come back to this topic and discuss it further.

Step 4 - Objective Function

This is the measure we want to optimize, that is, minimize (if it is an error function) or maximize if it is a reward function. The objective function can take many forms, for example, the **least squares error** or **mean least squares error**.

$$J(\mathbf{w}) = \frac{1}{2} \left\| \mathbf{t} - \mathbf{y} \right\|_2^2$$

- Note that $J(\mathbf{w})$ is a scalar value.
- This objective function is minimizing the error of projection (orthogonal distance to the curve).
- We can rewrite the objective function compactly in matrix/vector form:

$$J(\mathbf{w}) = \frac{1}{2} \left\| \mathbf{t} - \mathbf{X}\mathbf{w} \right\|_2^2 = \frac{1}{2} \left(\mathbf{t} - \mathbf{X}\mathbf{w} \right)^T \left(\mathbf{t} - \mathbf{X}\mathbf{w} \right)$$

where $\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^M \\ 1 & x_2 & x_2^2 & \dots & x_2^M \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^M \end{bmatrix}$, $\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_M \end{bmatrix}$, and $\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_N \end{bmatrix}$

- Recall that $\left\| \mathbf{z} \right\|_p$ is called the p -norm of the vector $\mathbf{z}$ and is defined as $\left(z_1^p + z_2^p + \dots + z_N^p \right)^{\frac{1}{p}}$.

Step 5 - Learning Algorithm

We fit the polynomial function model such that the *objective function* $J(\mathbf{w})$ is optimized.

$$J(\mathbf{w}) = \frac{1}{2} \left\| \mathbf{t} - \mathbf{X}\mathbf{w} \right\|_2^2$$

The optimization problem is posed as: $\arg_{\mathbf{w}} \min J(\mathbf{w})$

This reads "find the argument $\mathbf{w}$ that minimizes the objective function $J(\mathbf{w})$ ".

- What is the derivative of a scalar with respect to a vector?

$$\begin{aligned} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} &= \left[\frac{\partial J(\mathbf{w})}{\partial w_0}, \frac{\partial J(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial J(\mathbf{w})}{\partial w_M} \right]^T \end{aligned}$$

- If we rewrite the objective function as:

$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{2} \left(\mathbf{t} - \mathbf{X}\mathbf{w} \right)^T \left(\mathbf{t} - \mathbf{X}\mathbf{w} \right) \\ &= \frac{1}{2} \left(\mathbf{t}^T - \mathbf{w}^T \mathbf{X}^T \right) \left(\mathbf{t} - \mathbf{X}\mathbf{w} \right) \\ &= \frac{1}{2} \left(\mathbf{t}^T - \mathbf{w}^T \mathbf{X}^T \right) \left(\mathbf{t} - \mathbf{X}\mathbf{w} \right) \\ &= \frac{1}{2} \left(\mathbf{t}^T \mathbf{t} - \mathbf{t}^T \mathbf{X}\mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{t} + \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w} \right) \end{aligned}$$

- Solving for $\mathbf{w}$, we find:

[illegible]

$$\begin{aligned} \mathbf{X}^T \mathbf{t} - \mathbf{X}^T \mathbf{w} &= \\ \left(\mathbf{X}^T \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{t} \end{aligned}$$

- This gives us the optimal set of parameters $\mathbf{w}$ that minimize the objective function $J(\mathbf{w})$.

Test Data

After the model is trained (i.e. complete optimization of error function using the training labeled data), the **goal** is to *predict* the output values to *new*, unseen and unlabeled test data.

The steps in the test data are:

- Step 1: Extract (the same) features
- Step 2: Run through the trained model using the optimal set of parameters $\mathbf{w}$ to compute the output prediction value

Code Implementation

What can you control?

- Feature vectors or *basis functions*, and
- Model order, M .

Example: Input Data is sampled from a (noisy) sine curve

Suppose our data comes from a noisy sinusoidal: $t = \sin(2\pi x) + \epsilon$ where ϵ is a (univariate) Gaussian zero-mean random noise.

- The univariate Gaussian Distribution is defined as:

$$\mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi \sigma^2}} \exp\left\{ -\frac{(x - \mu)^2}{2\sigma^2} \right\}$$

where μ is the mean and σ^2 is the variance.

- If the noise is zero-mean Gaussian distributed, it is like we are saying there is a Gaussian around the true curve:

$$t = y + \epsilon$$

Let's generate data from the *true* underlying function (which, in practice, we would not know).

```
In [8]: def NoisySinusoidalData(N, a, b, sigma):
        '''Generates N data points in the range [a,b) sampled from a  $\sin(2\pi x)$ 
        with additive zero-mean Gaussian random noise with standard deviation  $\sigma$ 

        # N input samples, evenly spaced numbers between [a,b) incrementing by 1
        x = np.linspace(a,b,N)

        # draw N sampled from a univariate Gaussian distribution with mean 0, si
        noise = np.random.normal(0,sigma,N)

        # desired values, noisy sinusoidal
        t = np.sin(2*np.pi*x) + noise

        return x, t
```

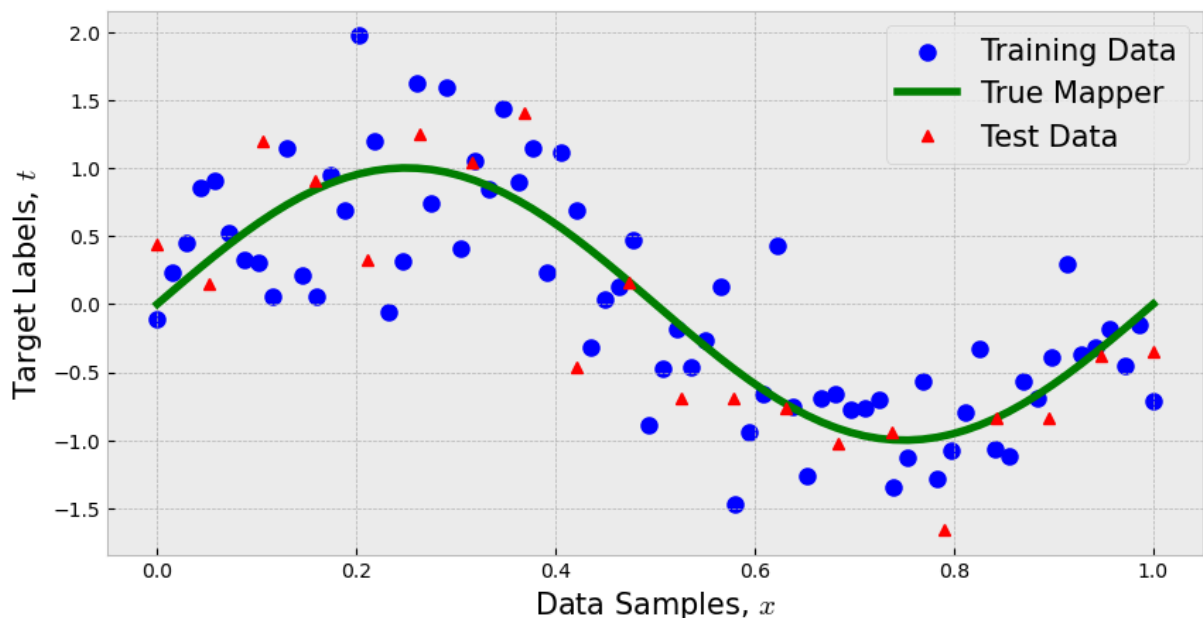
```
In [9]: # Generate input samples and desired values
N_train = 70 # number of data samples for training
N_test = 20 # number of data samples for test

a, b = [0,1] # data samples interval

sigma_train = 0.4 # standard deviation of the zero-mean Gaussian noise -- tr
sigma_test = 0.5 # standard deviation of the zero-mean Gaussian noise -- tes

x_train, t_train = NoisySinusoidalData(N_train, a, b, sigma_train) # Trainin
x_true, t_true = NoisySinusoidalData(N_train, a, b, 0) # True Sinusoidal - i
x_test, t_test = NoisySinusoidalData(N_test, a, b, sigma_test) # Test Data -
```

```
In [10]: plt.figure(figsize=(10,5))
plt.scatter(x_train, t_train, c='b', linewidths=3, label = 'Training Data')
plt.plot(x_true, t_true, 'g', linewidth=4, label = 'True Mapper')
plt.plot(x_test, t_test, 'r^', label = 'Test Data')
plt.legend(fontsize=15)
plt.xlabel('Data Samples,  $x$ ',size=15)
plt.ylabel('Target Labels,  $t$ ',size=15);
```



Now let's fit the data using the *polynomial regression* model:

```
In [11]: M=3 # hard-coding the hyperparameter to 3 for now (polynomial model order)
```

```
In [13]: x_train.shape
```

```
Out[13]: (70,)
```

```
In [14]: x_train[1]
```

```
Out[14]: 0.014492753623188406
```

```
In [15]: [x_train[1]**i for i in range(M+1)]
```

```
Out[15]: [1.0, 0.014492753623188406, 0.00021003990758244068, 3.044056631629575e-06]
```

```
In [16]: np.array([x_train[1]**i for i in range(M+1)])
```

```
Out[16]: array([1.00000000e+00, 1.44927536e-02, 2.10039908e-04, 3.04405663e-06])
```

```
In [17]: np.array([x_train**i for i in range(M+1)])
```

```
Out[17]: array([[1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00, 1.00000000e+00, 1.00000000e+00,
    1.00000000e+00, 1.00000000e+00],
 [0.00000000e+00, 1.44927536e-02, 2.89855072e-02, 4.34782609e-02,
   5.79710145e-02, 7.24637681e-02, 8.69565217e-02, 1.01449275e-01,
   1.15942029e-01, 1.30434783e-01, 1.44927536e-01, 1.59420290e-01,
   1.73913043e-01, 1.88405797e-01, 2.02898551e-01, 2.17391304e-01,
   2.31884058e-01, 2.46376812e-01, 2.60869565e-01, 2.75362319e-01,
   2.89855072e-01, 3.04347826e-01, 3.18840580e-01, 3.33333333e-01,
   3.47826087e-01, 3.62318841e-01, 3.76811594e-01, 3.91304348e-01,
   4.05797101e-01, 4.20289855e-01, 4.34782609e-01, 4.49275362e-01,
   4.63768116e-01, 4.78260870e-01, 4.92753623e-01, 5.07246377e-01,
   5.21739130e-01, 5.36231884e-01, 5.50724638e-01, 5.65217391e-01,
   5.79710145e-01, 5.94202899e-01, 6.08695652e-01, 6.23188406e-01,
   6.37681159e-01, 6.52173913e-01, 6.66666667e-01, 6.81159420e-01,
   6.95652174e-01, 7.10144928e-01, 7.24637681e-01, 7.39130435e-01,
   7.53623188e-01, 7.68115942e-01, 7.82608696e-01, 7.97101449e-01,
   8.11594203e-01, 8.26086957e-01, 8.40579710e-01, 8.55072464e-01,
   8.69565217e-01, 8.84057971e-01, 8.98550725e-01, 9.13043478e-01,
   9.27536232e-01, 9.42028986e-01, 9.56521739e-01, 9.71014493e-01,
   9.85507246e-01, 1.00000000e+00],
 [0.00000000e+00, 2.10039908e-04, 8.40159630e-04, 1.89035917e-03,
   3.36063852e-03, 5.25099769e-03, 7.56143667e-03, 1.02919555e-02,
   1.34425541e-02, 1.70132325e-02, 2.10039908e-02, 2.54148288e-02,
   3.02457467e-02, 3.54967444e-02, 4.11678219e-02, 4.72589792e-02,
   5.37702163e-02, 6.07015333e-02, 6.80529301e-02, 7.58244066e-02,
   8.40159630e-02, 9.26275992e-02, 1.01659315e-01, 1.11111111e-01,
   1.20982987e-01, 1.31274942e-01, 1.41986978e-01, 1.53119093e-01,
   1.64671288e-01, 1.76643562e-01, 1.89035917e-01, 2.01848351e-01,
   2.15080865e-01, 2.28733459e-01, 2.42806133e-01, 2.57298887e-01,
   2.72211720e-01, 2.87544633e-01, 3.03297627e-01, 3.19470699e-01,
   3.36063852e-01, 3.53077085e-01, 3.70510397e-01, 3.88363789e-01,
   4.06637261e-01, 4.25330813e-01, 4.44444444e-01, 4.63978156e-01,
   4.83931947e-01, 5.04305818e-01, 5.25099769e-01, 5.46313800e-01,
   5.67947910e-01, 5.90002100e-01, 6.12476371e-01, 6.35370720e-01,
   6.58685150e-01, 6.82419660e-01, 7.06574249e-01, 7.31148918e-01,
   7.56143667e-01, 7.81558496e-01, 8.07393405e-01, 8.33648393e-01,
   8.60323461e-01, 8.87418610e-01, 9.14933837e-01, 9.42869145e-01,
   9.71224533e-01, 1.00000000e+00],
 [0.00000000e+00, 3.04405663e-06, 2.43524531e-05, 8.21895291e-05,
   1.94819624e-04, 3.80507079e-04, 6.57516232e-04, 1.04411142e-03,
```

```
1.55855700e-03, 2.21911728e-03, 3.04405663e-03, 4.05163938e-03,
5.26012986e-03, 6.68779242e-03, 8.35289140e-03, 1.02736911e-02,
1.24684560e-02, 1.49554502e-02, 1.77529383e-02, 2.08791844e-02,
2.43524531e-02, 2.81910085e-02, 3.24131150e-02, 3.70370370e-02,
4.20810389e-02, 4.75633849e-02, 5.35023394e-02, 5.99161667e-02,
6.68231312e-02, 7.42414972e-02, 8.21895291e-02, 9.06854911e-02,
9.97476477e-02, 1.09394263e-01, 1.19643602e-01, 1.30513928e-01,
1.42023506e-01, 1.54190601e-01, 1.67033475e-01, 1.80570395e-01,
1.94819624e-01, 2.09799427e-01, 2.25528068e-01, 2.42023811e-01,
2.59304920e-01, 2.77389661e-01, 2.96296296e-01, 3.16043092e-01,
3.36648311e-01, 3.58130219e-01, 3.80507079e-01, 4.03797156e-01,
4.28018715e-01, 4.53190019e-01, 4.79329333e-01, 5.06454922e-01,
5.34585049e-01, 5.63737980e-01, 5.93931978e-01, 6.25185307e-01,
6.57516232e-01, 6.90943018e-01, 7.25483929e-01, 7.61157229e-01,
7.97981182e-01, 8.35974052e-01, 8.75154105e-01, 9.15539605e-01,
9.57148815e-01, 1.00000000e+00]])
```

```
In [18]: np.array([x_train**i for i in range(M+1)]).shape
```

```
Out[18]: (4, 70)
```

```
In [19]: np.array([x_train**i for i in range(M+1)]).T.shape
```

```
Out[19]: (70, 4)
```

```
In [20]: # feature matrix
X = np.array([x_train**i for i in range(M+1)]).T

X.shape
```

```
Out[20]: (70, 4)
```

```
In [21]: def PolynomialRegression(x, t, M):
    '''Fit a polynomial of order M to the data input data x and desire value

    # Compute feature matrix X with polynomial features
    X = np.array([x**i for i in range(M+1)]).T

    # Compute the solution for the parameters w
    w = np.linalg.inv(X.T@X)@X.T@t
    # @ symbols are for matrix multiplications

    # Compute model prediction
    y = X@w

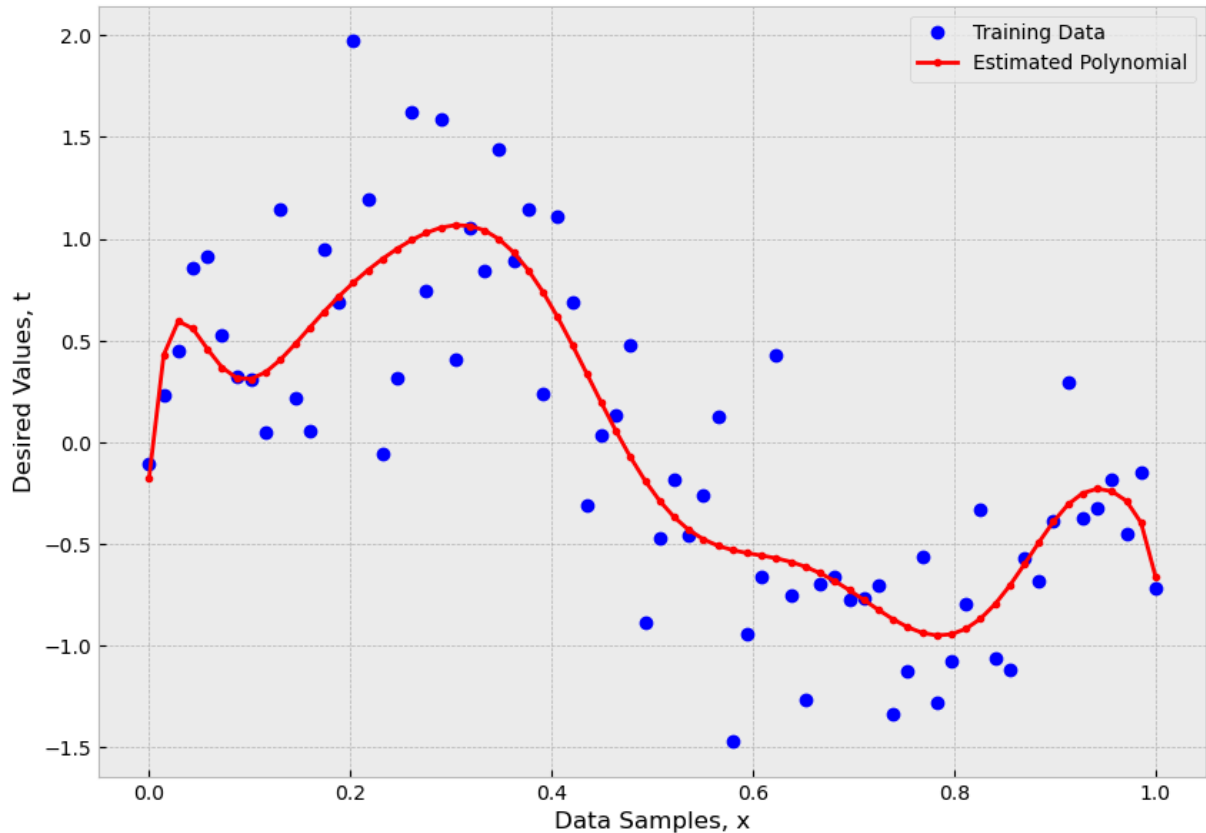
    return w, y
```

```
In [29]: # Model Order
M = 20

# Find the parameters that fit the noisy sinusoidal
w, y_train = PolynomialRegression(x_train, t_train, M)

plt.figure(figsize=(10,7))
plt.plot(x_train,t_train,'bo', label='Training Data')
```

```
plt.plot(x_train,y_train,'.-r', label = 'Estimated Polynomial')
# plt.plot(x_true,t_true,'g', label = 'True Function')
plt.legend()
plt.xlabel('Data Samples, x')
plt.ylabel('Desired Values, t');
```



Let's see how well does this trained model **generalize** to the test data, to which we do not have labels.

In the test data:

- Apply the same feature extraction as in training: $\mathbf{X}_{\text{test}} = \Phi(\mathbf{x}_{\text{test}})$, where $\mathbf{X}_{\text{test}}$ is a $K \times (M+1)$ data matrix
- Predict the output using $\mathbf{w}^*$: $\mathbf{y}_{\text{test}} = \Phi(\mathbf{x}_{\text{test}})\mathbf{w}^*$

```
In [30]: def PolynomialRegression_test(x_test, w):

    # Compute test feature matrix
    X = np.array([x_test**i for i in range(len(w))]).T

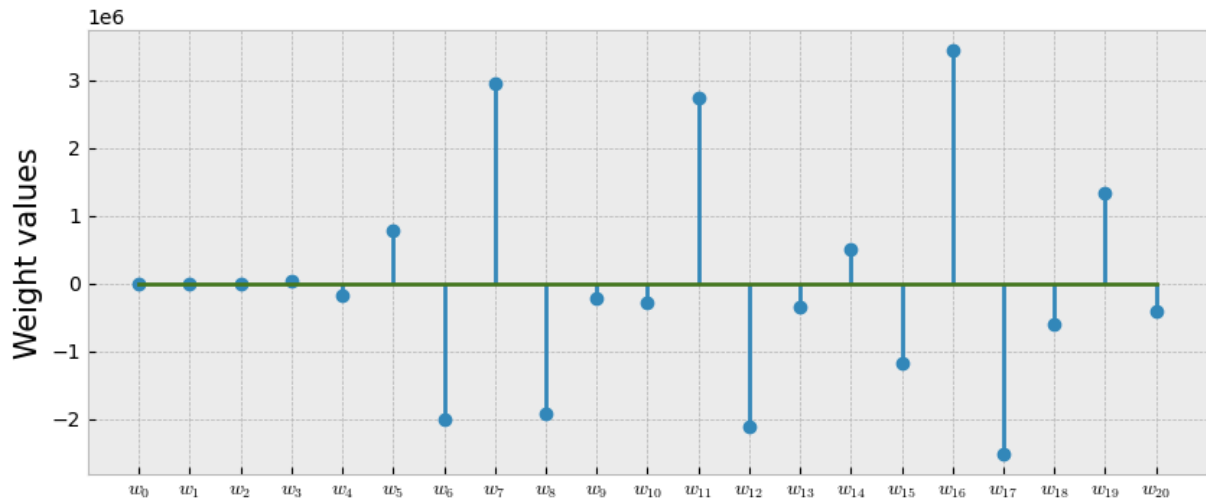
    # Prediction for test samples using the trained model
    y = X@w

    return y
```

```
In [31]: # Test set predictions
y_test = PolynomialRegression_test(x_test, w)

plt.figure(figsize=(10,7))
plt.plot(x_train, t_train, 'bo', label='Training Data')
plt.plot(x_train, y_train, '-.r', label = 'Estimated Polynomial in Training')
plt.plot(x_true, t_true, 'g', label = 'True Function')
plt.plot(x_test, t_test, 'r^', label = 'Test Data')
plt.plot(x_test, y_test, '--k', label = 'Prediction in Test')
plt.legend()
plt.xlabel('Data Samples, x')
plt.ylabel('Desired Values, t');
```

- How do the weights look like?

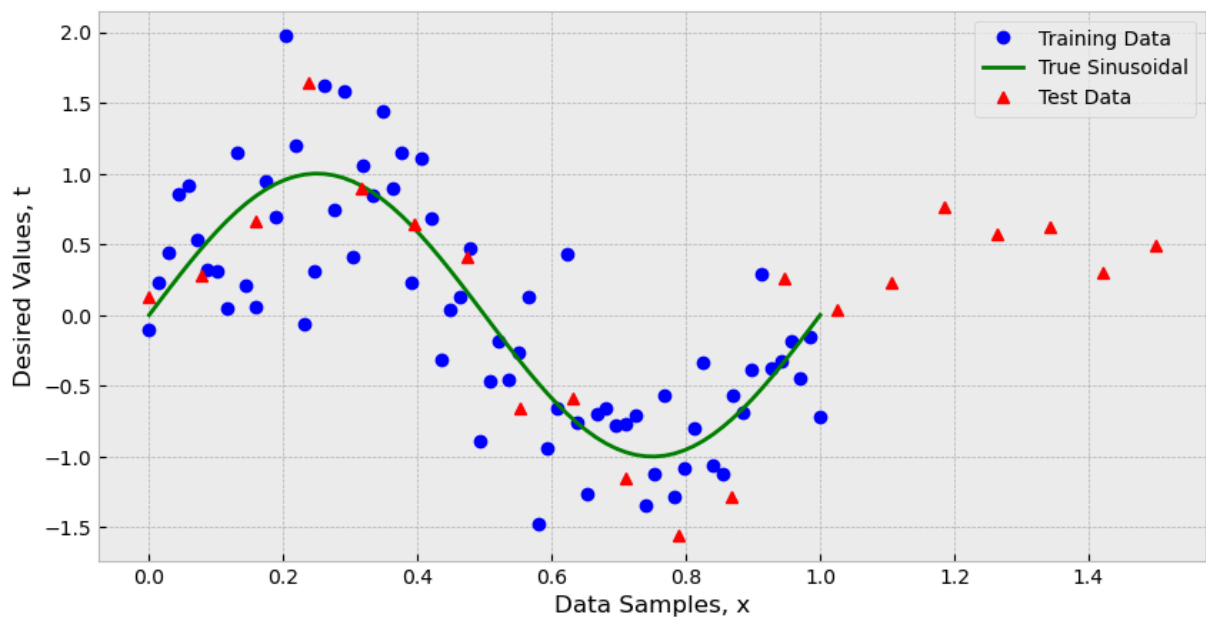


- What happens when the test points fall outside the range of what the model has *learned*?

```
In [33]: M = 4
w, y_train = PolynomialRegression(x_train, t_train, M)

x_test2, t_test2 = NoisySinusoidalData(N_test, 0, 1.5, sigma_test)

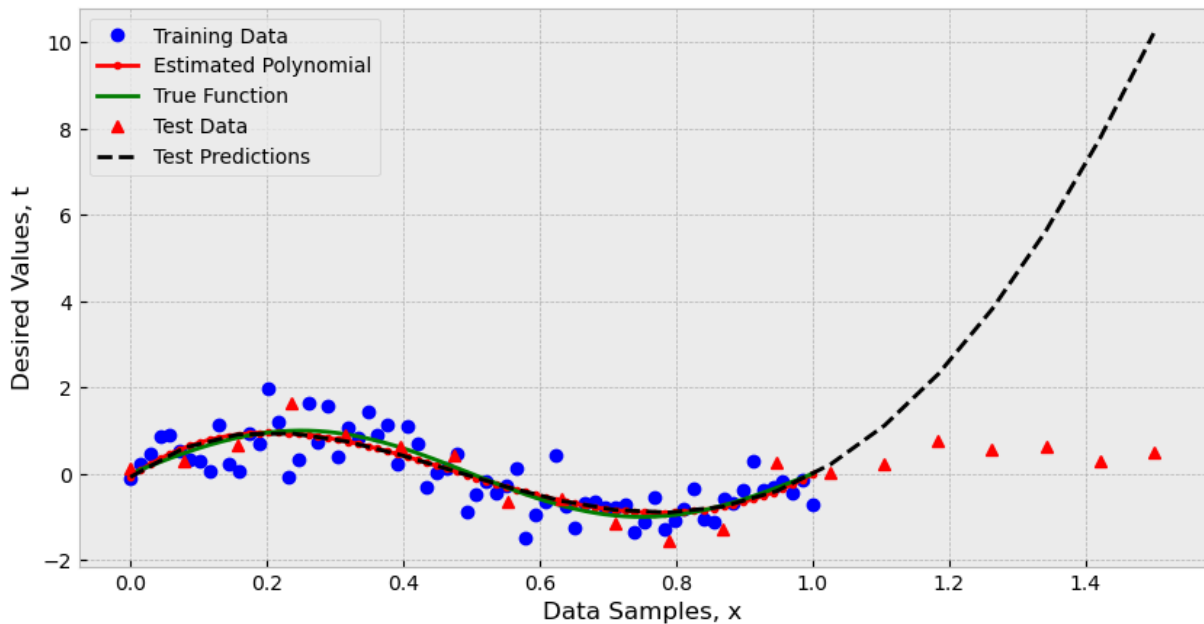
plt.figure(figsize=(10,5))
plt.plot(x_train, t_train, 'bo', label = 'Training Data')
plt.plot(x_true, t_true, '-g', label = 'True Sinusoidal')
plt.plot(x_test2, t_test2, 'r^', label = 'Test Data')
plt.legend()
plt.xlabel('Data Samples, x')
plt.ylabel('Desired Values, t');
```



```
In [34]: y_test2 = PolynomialRegression_test(x_test2, w)

plt.figure(figsize=(10,5))
```

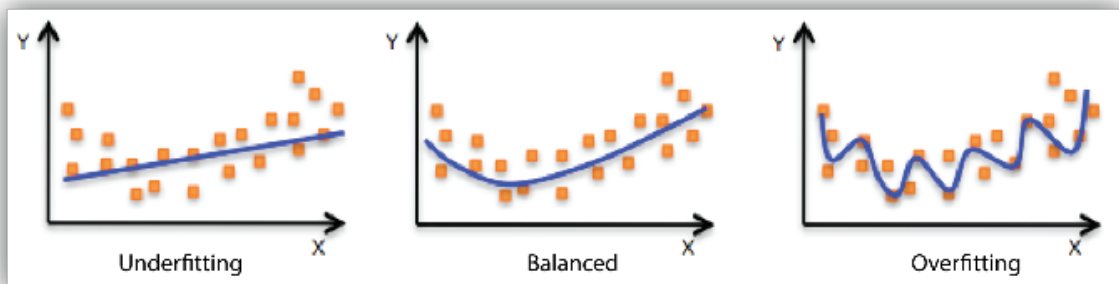
```
plt.plot(x_train,t_train,'bo', label='Training Data')
plt.plot(x_train,y_train,'.-r', label = 'Estimated Polynomial')
plt.plot(x_true,t_true,'-g', label = 'True Function')
plt.plot(x_test2,t_test2,'r^', label = 'Test Data')
plt.plot(x_test2,y_test2,'--k', label = 'Test Predictions')
plt.legend()
plt.xlabel('Data Samples, x')
plt.ylabel('Desired Values, t');
```



Overfitting and Underfitting

In [8]: `Image('figures/fits.png',width=700)`

Out[8]:



Overfitting

In model fitting, a model is said to have **overfit** to the training data, if it is *not* able to *generalize* to new and unknown data. We can observe this effect as the training error is small but the error in the test data is very large.

In other words, overfitting means that the *true* underlying model of the data is not estimated/learned properly, instead the model returned a poor representation that **memorized** the training samples **and** any meaningless noise in the data.

Underfitting

In model fitting, a model is said to have **underfit** if the error in both in training and test data sets are still decreasing.

An underfitted model is not complex enough to model all the characteristics in the data.

As we saw before, as the polynomial model order increases, the weight values become very large! This is a good indication that we may be *memorizing* the training data.

- As M increases, there are more parameters (more elements in the vector $\mathbf{w}$) to learn and, so, the model becomes more complex.
- The model order M in polynomial regression regulates the model *complexity*. The large M is, the model complex the model is.
 - In a Polynomial regression it also controls the richness of the feature space.
- **Occam's Razor** states that the simplest model that works for our data is usually the most appropriate and sufficient. Model simplicity can mean different things, but we can consider model order.

As a model that is too *complex*, it is more likely to *overfit*.

- This essentially means it "memorizes" the input training data and becomes incapable of predicting outside it,
 - This includes memorizing all of the training data's noise!
-